

ICDFA PRACTICAL LAB REPORT

Course: Intro to Critical Infrastructure for IT Learners

Lab Assignment: LAB 01 — Introduction to Programmable Logic Controllers

Student Name: WENDELL TWUMASI

Registration Number: C11/26/ACIS/17355

Submission Date: September 21, 2026

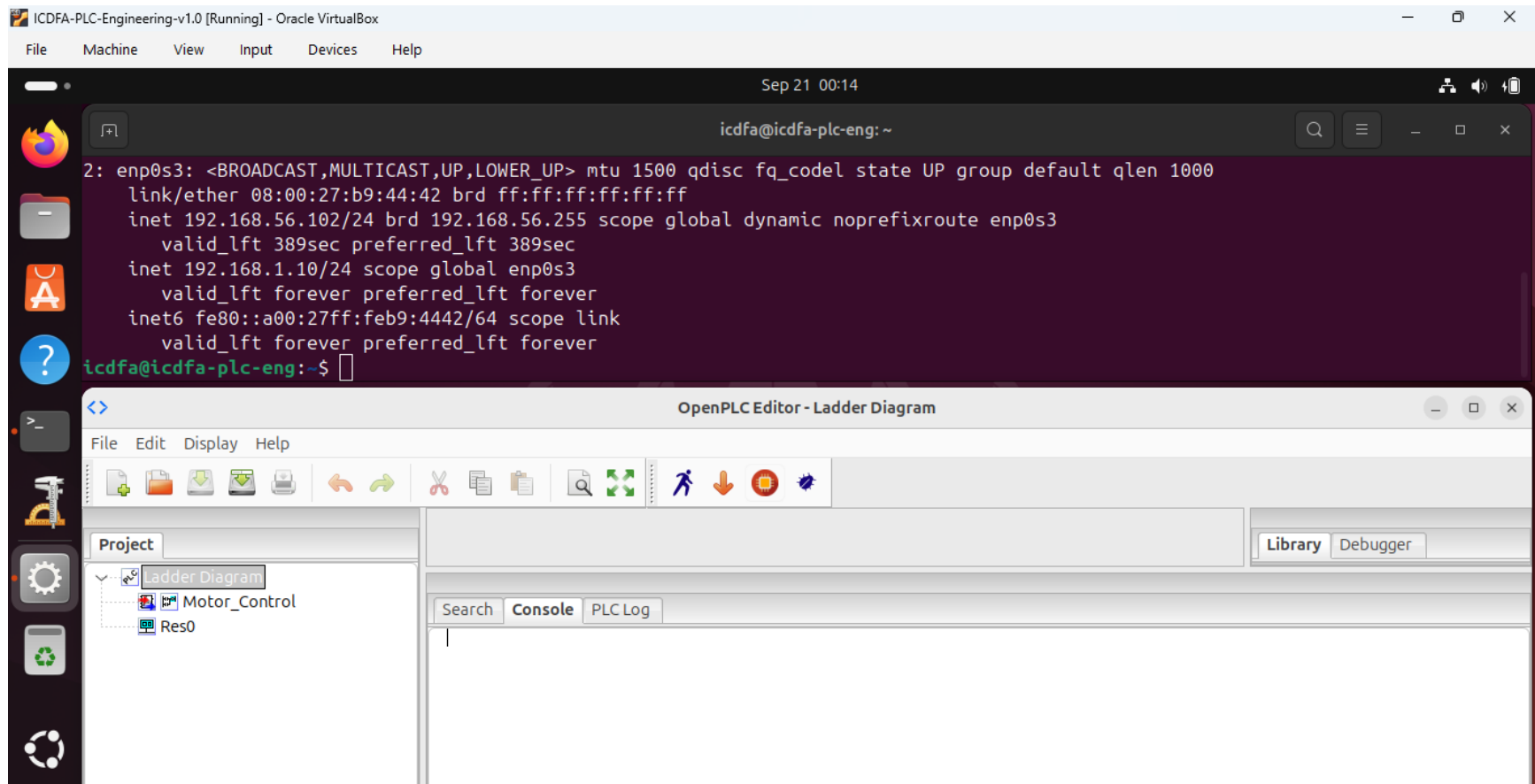
SCREENSHOT 1 — Laboratory Environment and PLC Identification

1. Target System Configuration & Environment Details

To meet the laboratory access criteria, the ICDFA PLC Virtual Machine was deployed in an isolated network environment.

- **Virtual Machine Name:** ICDFA-PLC-Engineering-v1.0
- **Operating System:** Ubuntu Linux (Custom ICDFA Industrial Workstation Edition)
- **Verified IP Address:** 192.168.1.10/24 (*Interface enp0s3*)
- **User Account:** icdfa
- **Network Mode:** Host-Only / Internal Isolated Network
- **Primary Application:** OpenPLC Editor

2. Screenshot 1 — Operational PLC Environment



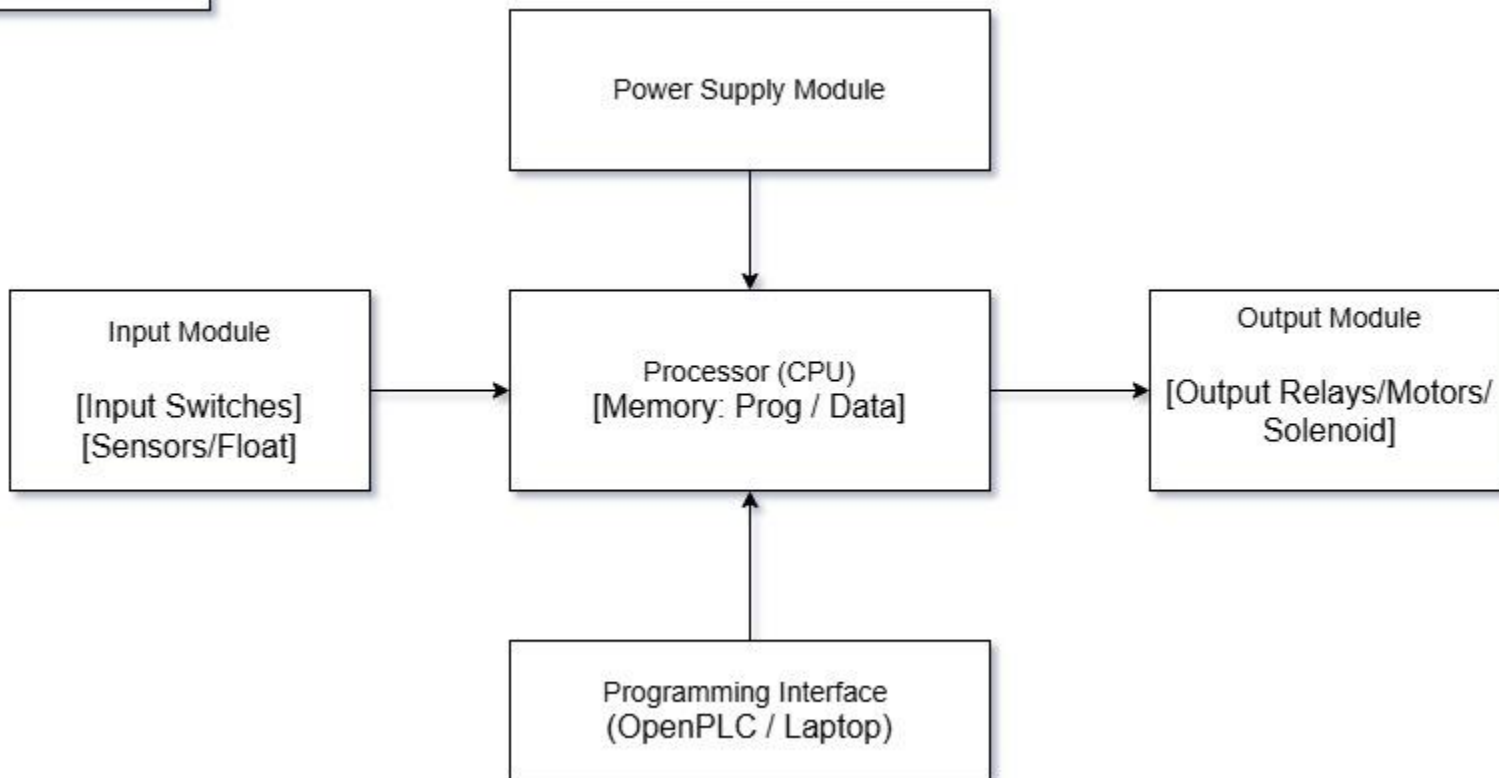
Caption: Figure 1.1 — Operational ICDFA PLC Virtual Machine desktop displaying terminal IP verification (192.168.1.10/24) alongside the launched OpenPLC Editor application.

SCREENSHOT 2 — PLC Architecture Analysis

1. Overview of PLC Architecture

A Programmable Logic Controller (PLC) is an industrial-grade micro-computer designed to execute automated control logic in critical infrastructure environments. Unlike general-purpose computers, a PLC is modularly constructed to interface directly with sensors, actuators, and power circuits.

WENDELL TWUMASI



2. Detailed Component Breakdown

- **Processor / CPU (Central Processing Unit):** Acts as the central computing unit of the PLC. It reads input states from data registers, evaluates user-programmed Ladder Logic, and writes execution commands to output modules. The CPU runs in a continuous scan loop: *Input Scan* → *Logic Execution* → *Output Scan*.
- **Memory:** Divided into the Program Memory and the Data Memory. The Program Memory stores the compiled application logic (e.g., ladder diagrams). The Data Memory stores I/O status tables, variable's values (BOOL, INT, REAL), and the accumulations of the timer and counter, and the system registers. .
- **Power Supply Module:** Converts high-voltage AC or main DC power (e.g., 120/230V AC or 24V DC) into regulated low-voltage DC power required to run the CPU, backplane bus, and internal module electronics.
- **Input Module:** Acts as the interface between field sensing devices and the CPU. It receives physical electrical signals from switches or sensors, provides optical isolation to protect sensitive electronics, and converts the signals into digital logic data (0 or 1) for the processor.
- **Output Module:** Serves as the interface between the CPU and field load devices. It converts low-power logic instructions from the CPU into higher-voltage/current control signals to drive relays, motor starters, valves, or indicator lights.
- **Rack / Chassis:** The physical frame and printed circuit backplane that mechanically holds and electrically interconnects the CPU, power supply, and I/O modules. It includes the high-speed data bus for inter-module communication.
- **Programming Interface / Device:** An external computer or software suite (such as **OpenPLC Editor**) used by technicians to write, compile, debug, and upload programs to the PLC memory. Once uploaded, the PLC executes the logic autonomously.

Evidence 3 — Input/Output Classification

1. Device Classification Table

#	Device Name	Device Category	Signal Type	Industrial Process Application
1	Liquid Level Float Switch	Input	Discrete (Digital)	Detects when a fluid level reaches a maximum threshold in a storage tank.
2	Normally Open Push Button (Start)	Input	Discrete (Digital)	Operator input to initiate equipment startup.
3	RTD Temperature Sensor	Input	Analogue	Continuously measures process vessel temperatures across a 0°C to 150°C scale.
4	Motor Starter / Contactor	Output / Load	Discrete (Digital)	Switches electrical power to start or stop a heavy industrial conveyor motor.
5	Solenoid Valve	Output / Load	Discrete (Digital)	Electromechanically opens or closes a pneumatic valve to direct fluid flow.
6	Variable Frequency Drive (VFD)	Output / Load	Analogue	Adjusts pump motor speed dynamically via a 4--20 mA control signal.

2. Discrete (Digital) vs. Analogue I/O Explanation

- **Discrete (Digital) I/O:** Operates on a binary state system. The signal is restricted to one of two states at any given moment: **ON** (1 / High Voltage) or **OFF** (0 / Low Voltage). *Example:* A push button or float switch—it is either open (0) or closed (1).

- **Analogue I/O:** Works over a range of varying electrical values (such as 0"10"V" DC or 4"20" mA) over time. It represents physical values that varies continuously. Example: Pressure transmitter or temperature sensor indicating exact, varying values (e.g., 24.2°C, 24.5°C).

SCREENSHOT 4 — OpenPLC and Ladder Logic Practical

1. Screenshot 2 — OpenPLC Motor_Control Diagram

The screenshot displays the ICDFA-PLC-Engineering-v1.0 software interface within an Oracle VirtualBox window. The main workspace shows a ladder logic diagram for a motor control system. The diagram consists of two parallel normally open contacts labeled 'Stop' and 'Start', both of which are interlocked with a 'MotorOutput' coil. The 'Stop' contact is connected to the 'MotorOutput' coil, and the 'Start' contact is also connected to the 'MotorOutput' coil. The diagram is titled 'Motor_Control' and is part of a project named 'Res0'.

On the left, the 'Project' tree shows the 'Ladder Diagram' and 'Motor_Control' components. Below the project tree, a list of variables is shown: 'Stop (BOOL)', 'Start (BOOL)', and 'MotorOutput (BOOL)'. A search bar is located at the bottom left of the project tree.

The central panel displays the 'Motor_Control' configuration. It includes a 'Description' field, a 'Class Filter' set to 'All', and a table of variables:

#	Name	Class	Type	Location	Initial Value	Option	Documentation
2	Start	Local	BOOL				Start push button input
3	MotorOutp	Local	BOOL				Output coil to motor

Below the table, the 'Debug' section shows the configuration for 'Config0.Res0.instance0'. It displays a ladder logic diagram with the same 'Stop' and 'Start' contacts and 'MotorOutput' coil. The 'MotorOutput' coil is labeled 'MotorOutput'.

On the right, the 'Debugger' panel shows a 'Range' of '1s' and a 'Stop' button. Below this, a graph shows the 'Stop' signal over time, with a value of 1 from 0 to 1000.

The bottom panel contains a 'Search' bar and tabs for 'Console' and 'PLC Log'.

Caption: Figure 4.1 — OpenPLC Editor displaying the Motor_Control ladder diagram showing the variable table, two power rails, contacts, and output coil.

2. Variable Table & Element Mapping

Variable Name	Data Type	Ladder Contact / Coil Symbol	Physical Function
Stop	BOOL	Normally Closed Contact] [	Stop push button input. Allows rung continuity when unpressed (TRUE).
Start	BOOL	Normally Open Contact] [	Start push button input. Blocks continuity until pressed (TRUE).
MotorOutput	BOOL	Parallel NO Contact] [	Latching / Seal-in contact bound to output state.
MotorOutput	BOOL	Output Coil ()	Controls physical motor contactor/relay load.

3. Detailed Explanation of Control Logic Execution

A. Circuit Topology

The logic consists of a single ladder rung connected between the **Left Power Rail** (24V DC) and the **Right Power Rail** (0V/Neutral):

1. The **Normally Closed Stop contact** is placed in series with the entire circuit on the left.
2. The **Normally Open Start contact** and the **Normally Open MotorOutput latch contact** are connected in parallel with each other.

3. The **MotorOutput coil** is connected on the far right end of the rung.

B. Step-by-Step Control Flow

1. **Idle State (Motor Stopped):** The Stop push button is not pressed. Since it is a Normally Closed () / () contact, it evaluates as TRUE (1) and allows power to pass through. The Start button is not pressed (Normally Open) (), and evaluates as FALSE (0). The MotorOutput coil is not energized, which means that the parallel MotorOutput contact is open (FALSE). There is no complete path from left rail to right rail. MotorOutput is OFF.
2. **Starting the Motor (Button Pressed):** The operator presses the momentary Start button. Start changes from FALSE to TRUE (1). Power flows from the Left Rail through Stop (TRUE), Start (TRUE), and into MotorOutput coil. MotorOutput coil then energizes to TRUE (1) and powers the motor.
3. **Latching / Seal-in Action (Button Released):** As soon as MotorOutput energizes, it's associated parallel contact MotorOutput) (closes (TRUE). When the operator releases the momentary Start button, Start opens back to FALSE. Power continues to flow through Stop (TRUE) and through the closed MotorOutput seal-in contact (TRUE), keeping the coil energized. Result: The motor continues to run continuously, even after the Start button has been released .
4. **Stopping the Motor (Stop Pressed):** The operator presses the Stop button. The Normally Closed Stop contact opens, evaluating to **FALSE (0)**. Path continuity across the entire rung is broken. **Result:** MotorOutput coil de-energizes (**FALSE**). The seal-in contact MotorOutput opens back to **FALSE**. When Stop is released, the circuit returns to the idle state.

Knowledge Check & Cybersecurity Reflection

1. Knowledge Check Responses

1. **What is the difference between an input module and an output module?**

Response: An input module condition external signals from field sensing devices (e.g., push buttons, sensors) to digital logic for the PLC CPU. An output module conditions the control logic signals from the CPU to power signals for operating field load devices (e.g., motors, solenoid valves).

Why is a normally closed contact commonly associated with stop or emergency-stop logic?

Response: Normally Closed (NC) contacts provide **failsafe operation**. Under normal operating conditions, current flows continuously through the closed contact. If a control wire breaks or power drops, the path automatically opens (FALSE), immediately halting equipment to prevent unsafe conditions.

2. Give one example of a discrete I/O signal and one example of an analog I/O signal.

Response:

- **Discrete I/O:** A liquid level switch returning 1 (ON) when full and 0 (OFF) when empty.
- **Analogue I/O:** A 4--20 mA signal from a VFD controlling motor speed between 0 and 1800 RPM.

3. What operational risk can arise when industrial automation devices become network-connected?

Response: Network connectivity exposes PLCs to cyber attacks, remote unauthorized access, and command injection. Attackers can tamper with ladder logic or spoof I/O telemetry, causing physical hardware damage, operational downtime, or severe safety hazards.

2. Cybersecurity Reflection: Network-Connected PLCs

Connecting Programmable Logic Controllers (PLCs) to industrial IP networks provides many benefits such as central monitoring, diagnostics, and remote programming which in-turn increases productivity. From a supervisory point of view, production schedules, field trends, and program changes can be easily viewed, adjusted, or updated without going to the PLCs cabinet. Overall, having a PLC connected to a network provides flexibility while reducing downtime improving manufacturing and production processes.

On the other hand, connecting PLCs to network switches breaks down their protection as stand-alone relay replacements, making them vulnerable to network-based attacks. For instance, most industrial PLC communication protocols such as Modbus and Ethernet/IP do not have built-in security features such as encryption and authentication mechanisms. A competent attacker who exploits an insecure PLC connection can easily modify a program in a PLC by manipulating input/output (I/O) data, compromising a facility's physical equipment, and creating hazardous situations such as water

treatment contamination. However, these attacks can be avoided by segmenting and protecting OT networks through gateways, industrial firewalls, packet analysis, and network access control.